Practical No. 06

Title of the Assignment: Implement DFS for Water Jug Problem

Problem statement: Use DFS for problem solving. Examples can be Water Jug Problem, Missionaries and Cannibals, Maze Solving, Blocks World etc.

Objective:

To understand the concept of Uninformed (Blind) search techniques.

To understand Depth First Search (DFS) for solving a problem.

To implement DFS for a problem-solving/planning problem and obtain the solution path.

Theory:

Depth First Search (DFS):

Depth First Search (DFS) is an uninformed search technique that explores a search space by going as deep as possible along one branch before backtracking and exploring another branch. DFS starts from the initial state, selects a successor state, and continues expanding the selected branch until the goal state is reached or no further valid state is available.

DFS can be implemented using a stack (LIFO – Last In, First Out) or by using recursion. In a problem-solving task, each state represents a possible configuration and each valid action generates a successor state. A visited set is used to avoid repeatedly exploring the same states when the state space contains cycles.

Key characteristics of DFS:

DFS is an uninformed (blind) search technique.

It explores one branch deeply before backtracking.

It uses a stack or recursion.

It does not require a heuristic function.

It may use less memory than BFS because it mainly stores the current search path and pending alternatives.

DFS does not necessarily find the shortest solution path.

DFS for Problem Solving:

DFS can be applied to problems such as the Water Jug Problem, Missionaries and Cannibals, Maze Solving and Blocks World. The problem is represented as a state-space search problem. The initial state is the starting configuration, the goal state is the required configuration, and operators/actions generate valid successor states.

DFS follows one possible sequence of actions until it reaches the goal or reaches a dead end. If a dead end is reached, the algorithm backtracks to the previous state and tries another available action.

Example – Water Jug Problem:

Consider two jugs of capacity 4 litres and 3 litres. Initially both jugs are empty. The objective is to obtain exactly 2 litres of water in one of the jugs. The possible actions include filling a jug, emptying a jug, and transferring water from one jug to another.

Initial State: (0, 0)

Goal State: (2, 0) or (0, 2)

A state is represented as (x, y), where x is the amount of water in the 4-litre jug and y is the amount of water in the 3-litre jug. DFS explores one sequence of valid operations deeply and backtracks whenever the selected sequence cannot lead to the goal.

Possible actions:

1. Fill the 4-litre jug.
2. Fill the 3-litre jug.
3. Empty the 4-litre jug.
4. Empty the 3-litre jug.
5. Pour water from the 4-litre jug into the 3-litre jug.
6. Pour water from the 3-litre jug into the 4-litre jug.

DFS Algorithm:

Step 1: Start.

Step 2: Define the initial state and the goal state.

Step 3: Create an empty stack and a visited set.

Step 4: Push the initial state onto the stack and mark it as visited.

Step 5: Pop the top state from the stack.

Step 6: If the selected state is the goal state, reconstruct and display the solution path, then stop.

Step 7: Otherwise, generate all valid successor states.

Step 8: For each unvisited successor state, store its parent and push it onto the stack.

Step 9: Continue exploring the deepest available state.

Step 10: If a branch reaches a dead end, backtrack by removing states from the current path and explore another branch.

Step 11: Repeat Steps 5–10 until the goal is reached or the stack becomes empty.

Step 12: If the stack becomes empty without finding the goal, report that no solution was found.

Step 13: Display the sequence of states/actions leading to the goal.

Step 14: Stop.

Advantages and Limitations of DFS:

Advantages:

Simple and easy to implement.

Requires relatively less memory than breadth-oriented search in many search spaces.

Can reach a deep solution without exploring all shallower states first.

Useful for maze solving, state-space exploration and planning problems.

Limitations:

It does not necessarily find the shortest solution.

It can get stuck exploring a deep branch unless visited-state checking or a depth limit is used.

Its performance depends on the order in which successor states are explored.

It may be inefficient for problems where the solution is close to the root but lies in a different branch.

Conclusion:

DFS was studied as an uninformed search technique for problem solving. The Water Jug Problem was considered as an example of state-space search, where valid operations generate successor states. DFS explores one branch deeply and uses backtracking to examine alternative branches. The solution path can be obtained by maintaining parent information for the generated states.

Oral Questions:

1. What is Depth First Search (DFS)?
2. Is DFS an informed or uninformed search technique?
3. Which data structure is used by DFS?
4. What is the difference between DFS and BFS?
5. How is DFS applied to the Water Jug Problem?
6. What is backtracking in DFS?
7. Does DFS always find the shortest path?
8. Why is a visited set required in state-space search?
9. What are the applications of DFS in AI?
10. What are the limitations of DFS?